

LAPORAN PRAKTIKUM
APLIKASI BERBASIS PLATFORM

MODUL 12 13

Tugas 5



Universitas
Telkom

Disusun oleh:

Yasvin Syahgana

2311102065

S1 IF-11-06

Dosen Pengampu:

Dedy Agung Prabowo, S.Kom., M.Kom

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM

2026

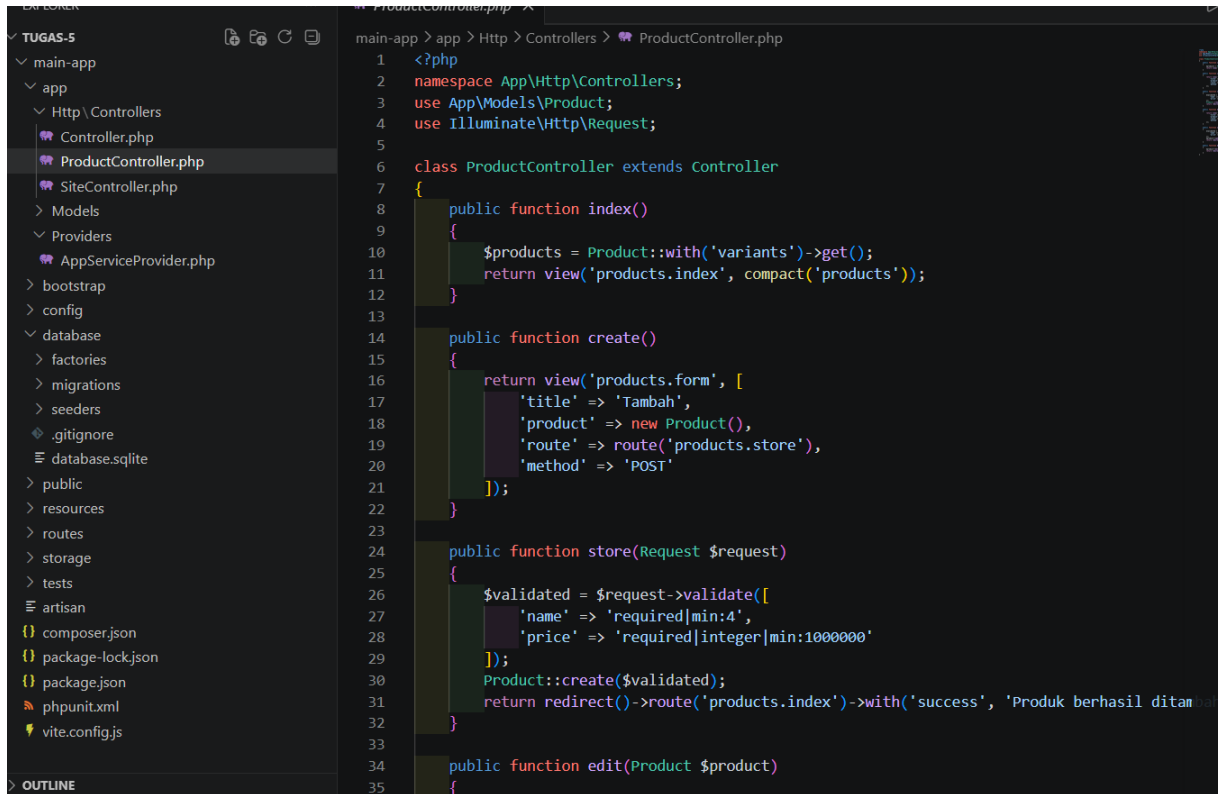
LINK GIT : <https://github.com/yasvinsyahgana/Laporan-Praktikum-ABP>

MODUL 12 13

Tugas 5

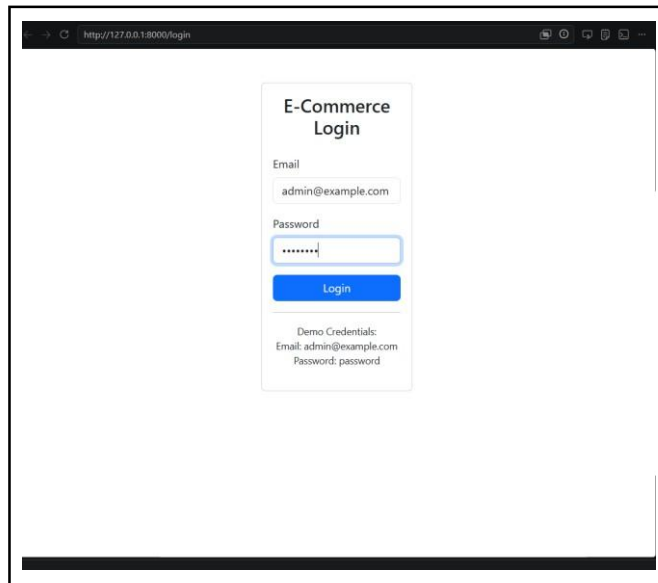
1. Product Controller

- Code (boleh copy/screenshot dengan syarat terlihat jelas dan tidak



```
1 <?php
2 namespace App\Http\Controllers;
3 use App\Models\Product;
4 use Illuminate\Http\Request;
5
6 class ProductController extends Controller
7 {
8     public function index()
9     {
10         $products = Product::with('variants')->get();
11         return view('products.index', compact('products'));
12     }
13
14     public function create()
15     {
16         return view('products.form', [
17             'title' => 'Tambah',
18             'product' => new Product(),
19             'route' => route('products.store'),
20             'method' => 'POST'
21         ]);
22     }
23
24     public function store(Request $request)
25     {
26         $validated = $request->validate([
27             'name' => 'required|min:4',
28             'price' => 'required|integer|min:1000000'
29         ]);
30         Product::create($validated);
31         return redirect()->route('products.index')->with('success', 'Produk berhasil ditam
32     }
33
34     public function edit(Product $product)
35     {
```

b. *Screenshot output*



c. Penjelasan dari code dan output (korelasi)

ProductController adalah pengendali yang ada dalam framework Laravel, berfungsi untuk mengatur informasi produk dengan menerapkan prinsip CRUD (Buat, Baca, Perbarui, Hapus). Pengendali ini menggunakan model Product guna berkomunikasi dengan basis data, serta Request untuk mengelola masukan dari pengguna. Pada metode `index()`, sistem akan mengambil seluruh informasi produk melalui `Product::all()` kemudian menampilkannya di halaman `products.index`. Metode `create()` dipakai untuk menunjukkan formulir penambahan produk dengan mengirimkan berbagai data seperti judul halaman, objek produk yang kosong, rute penyimpanan, dan metode HTTP POST.

Dalam tahap penyimpanan data, metode `store()` menerima masukan dari pengguna dan melakukan pemeriksaan agar nama produk minimal memiliki 4 karakter dan harga dalam bentuk angka dengan nilai minimal 1.000.000. Apabila pemeriksaan berhasil, data akan disimpan ke dalam basis data melalui `Product::create()`, selanjutnya pengguna akan diarahkan kembali ke halaman daftar produk dengan pesan sukses. Di samping itu, metode `show()` bertujuan untuk menampilkan rincian satu produk berdasarkan ID yang dipilih, sementara metode `edit()` berfungsi untuk menampilkan formulir pengeditan dengan data produk yang sudah ada, sehingga memungkinkan pengguna untuk melakukan perubahan pada informasi tersebut.

Method `update()` digunakan untuk memperbarui data produk yang telah dipilih. Pada method ini, Laravel kembali melakukan validasi input sebelum data diperbarui menggunakan `$product->update($validated)`. Setelah berhasil diperbarui, sistem akan mengarahkan pengguna kembali ke halaman daftar produk beserta notifikasi keberhasilan. Sementara itu, method `destroy()` digunakan untuk menghapus data produk dari database menggunakan `$product->delete()`. Secara keseluruhan, controller ini menjadi penghubung antara model, database, dan tampilan sehingga proses pengelolaan data produk dapat berjalan secara terstruktur dan efisien.

2. Site Controller

- a. Code (boleh copy/screenshot dengan syarat terlihat jelas dan tidak blur)

```
SiteController.php X
main-app > app > Http > Controllers > SiteController.php
1  <?php
2  namespace App\Http\Controllers;
3  use Illuminate\Http\Request;
4  use Illuminate\Support\Facades\Auth;
5
6  class SiteController extends Controller
7  {
8      public function showLogin()
9      {
10         if (Auth::check()) return redirect('/products');
11         return view('login');
12     }
13
14     public function auth(Request $request)
15     {
16         $credentials = $request->validate([
17             'email' => 'required|email',
18             'password' => 'required'
19         ]);
20
21         if (Auth::attempt($credentials)) {
22             $request->session()->regenerate();
23             return redirect()->intended('/products');
24         }
25         return back()->with('msg', 'Email atau password salah');
26     }
27
28     public function logout()
29     {
30         Auth::logout();
31         return redirect('/login');
32     }
33 }
34
```

b. *Screenshot output*

Admin

Logout

Form Tambah Produk

Nama

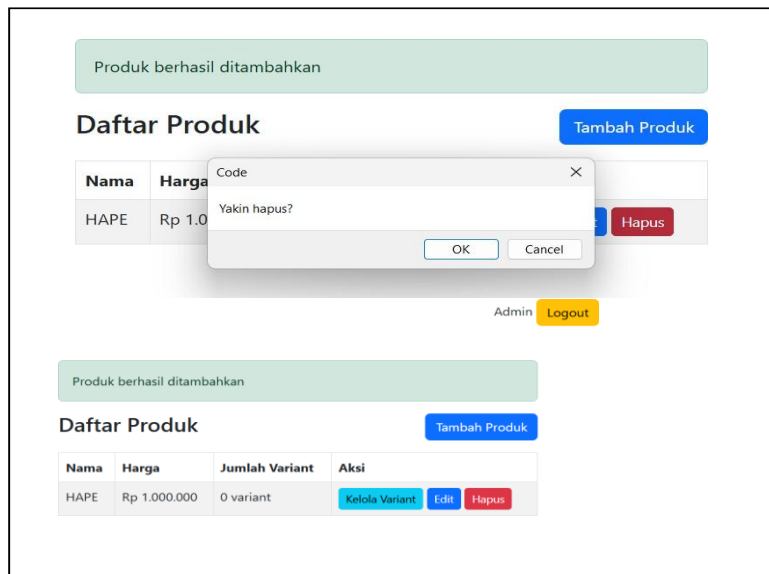
Es Teh

Harga

3000

Simpan

Batal



c. Penjelasan dari code dan output (korelasi)

file tampilan (view) Blade pada framework Laravel yang digunakan untuk menampilkan halaman daftar produk. Pada bagian awal, kode `@extends('template')` menunjukkan bahwa halaman ini menggunakan layout utama bernama `template`, sedangkan `@section('title', 'Daftar Produk')` digunakan untuk mengatur judul halaman menjadi “Daftar Produk”. Selanjutnya, seluruh isi halaman ditempatkan di dalam `@section('content')`. Struktur tampilan menggunakan komponen Bootstrap seperti `container`, `row`, dan `col-md-10` agar halaman memiliki tampilan yang rapi dan responsif. Kode juga memeriksa apakah terdapat pesan sukses menggunakan `session('success')`; jika ada, maka pesan akan ditampilkan dalam bentuk alert berwarna hijau sebagai notifikasi bahwa suatu proses berhasil dilakukan.

Pada bagian utama halaman, terdapat judul “Daftar Produk” dan tombol “Tambah Produk” yang mengarah ke route `products.create` untuk membuka form penambahan data produk baru. Data produk ditampilkan dalam bentuk tabel dengan kolom Nama, Harga, Jumlah Variant, dan Aksi. Pengulangan

data dilakukan menggunakan `@foreach($products as $product)` sehingga setiap produk yang dikirim dari controller akan ditampilkan secara otomatis. Nama produk ditampilkan menggunakan `{{ $product->name }}`, sedangkan harga diformat menjadi format rupiah menggunakan fungsi `number_format()`. Pada kolom jumlah variant, kode `{{ $product->variants()->count() }}` digunakan untuk menghitung total varian yang dimiliki setiap produk berdasarkan relasi model.

Pada kolom aksi, tersedia beberapa tombol untuk mengelola data produk. Tombol “Kelola Variant” digunakan untuk membuka halaman pengelolaan varian produk melalui route `variants.index`, sedangkan tombol “Edit” mengarah ke halaman edit produk menggunakan route `products.edit`. Selain itu, terdapat form penghapusan produk yang menggunakan method POST dengan tambahan `@method('DELETE')` agar Laravel mengenalinya sebagai request DELETE. Token `@csrf` digunakan sebagai perlindungan keamanan terhadap serangan CSRF. Sebelum data dihapus, sistem akan menampilkan konfirmasi melalui `onsubmit="return confirm('Yakin hapus?')"` sehingga pengguna harus menyetujui terlebih dahulu sebelum produk benar-benar dihapus dari database.

3. Middleware

a. Code (boleh copy/screenshot dengan syarat terlihat jelas dan tidak blur)

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;

class Authenticate
{
    public function handle(Request $request, Closure $next): mixed
    {
        if (! Auth::check()) {
            return redirect('/login');
        }
    }
}
```

```

        return $next($request);
    }
}

```

b. *Screenshot output*

c. Penjelasan dari code dan output (korelasi)

Middleware Authenticate pada framework Laravel yang berfungsi untuk membatasi akses pengguna terhadap halaman tertentu dalam aplikasi. Middleware ini berada pada folder `App\Http\Middleware` dan digunakan untuk memeriksa apakah pengguna sudah melakukan login atau belum sebelum dapat mengakses suatu route atau halaman. Pada bagian awal kode, terdapat `import Request` untuk menangani request dari pengguna, Closure untuk melanjutkan proses request berikutnya, serta `Auth` yang digunakan untuk memeriksa status autentikasi pengguna.

Method utama pada middleware ini adalah `handle(Request $request, Closure $next)`. Method ini akan dijalankan setiap kali pengguna mencoba mengakses route yang menggunakan middleware Authenticate. Pada bagian `if (! Auth::check())`, Laravel memeriksa apakah pengguna belum login. Fungsi `Auth::check()` akan menghasilkan nilai `true` jika pengguna sudah terautentikasi dan `false` jika belum login. Jika pengguna belum login, maka sistem akan langsung mengarahkan pengguna ke halaman `/login` menggunakan `redirect('/login')`.

Apabila pengguna sudah login, maka kode `return $next($request);` akan dijalankan. Fungsi ini berarti request akan diteruskan ke proses berikutnya,

seperti controller atau halaman yang dituju. Dengan adanya middleware ini, aplikasi menjadi lebih aman karena hanya pengguna yang telah berhasil melakukan autentikasi yang dapat mengakses halaman tertentu, misalnya halaman pengelolaan produk atau dashboard aplikasi.

4. Model Relasi

a. Code (boleh copy/screenshot dengan syarat terlihat jelas dan tidak blur)

```
<tr>
    <th>Nama</th>
    <th>Harga</th>
    <th>Jumlah Variant</th>
    <th>Aksi</th>
</tr>
</thead>
<tbody>
    @foreach($products as $product)
    <tr>
        <td>{{ $product->name }}</td>
        <td>Rp {{
number_format($product->price, 0, ',', '.') }}</td>
        <td>{{ $product->variants()-
>count() }} variant</td>
        <td>
            <a href="{{
route('variants.index', $product->id) }}" class="btn
btn-sm btn-info">Kelola Variant</a>
            <a href="{{
route('products.edit', $product->id) }}" class="btn
btn-sm btn-primary">Edit</a>
            <form method="POST"
action="{{ route('products.destroy', $product->id) }}"
style="display:inline" onsubmit="return confirm('Yakin
hapus?') ">
                @csrf
                @method('DELETE')
                <button class="btn btn-
sm btn-danger">Hapus</button>
            </form>
        </td>
    </tr>
    @endforeach
</tbody>
</table>
</div>
</div>
```

```
</div>
```

b. *Screenshot output*

The screenshot displays a web interface for managing products. At the top, there is a 'Form Tambah Produk' (Add Product Form) with two input fields: 'Nama' (Name) containing 'LAPTOP' and 'Harga' (Price) containing '5000'. The 'Harga' field has a red border and an error message below it: 'The price field must be at least 1000000.'. Below the form are two buttons: 'Simpan' (Save) in green and 'Batal' (Cancel) in grey. Below the form, a green alert box states 'Produk berhasil ditambahkan' (Product successfully added). Below the alert is a 'Daftar Produk' (Product List) table with columns: 'Nama', 'Harga', 'Jumlah Variant', and 'Aksi'. The table contains two rows: one for 'HAPE' with a price of 'Rp 1.000.000' and 0 variants, and one for 'LAPTOP' with a price of 'Rp 5.000.000' and 0 variants. Each row has three action buttons: 'Edit', 'Hapus', and 'Tambah Produk' (Add Product).

Nama	Harga	Jumlah Variant	Aksi
HAPE	Rp 1.000.000	0 variant	Edit Hapus Tambah Produk
LAPTOP	Rp 5.000.000	0 variant	Edit Hapus Tambah Produk

c. Penjelasan dari code dan output (korelasi)

Blade pada Laravel yang digunakan untuk menampilkan halaman daftar produk dalam bentuk tabel. Pada bagian awal, `@extends('template')` menunjukkan bahwa halaman ini menggunakan layout utama bernama `template`, sedangkan `@section('title', 'Daftar Produk')` digunakan untuk mengatur judul halaman menjadi “Daftar Produk”. Seluruh isi halaman ditempatkan di dalam `@section('content')`. Struktur tampilan menggunakan Bootstrap seperti `container`, `row`, dan `col-md-10` agar halaman terlihat rapi dan responsif. Selain itu, kode `@if(session('success'))` digunakan untuk memeriksa apakah terdapat pesan sukses dari proses sebelumnya, misalnya setelah menambah, mengedit, atau menghapus produk, lalu menampilkannya dalam bentuk alert berwarna hijau.

Pada bagian utama halaman terdapat judul “Daftar Produk” dan tombol “Tambah Produk” yang mengarah ke route `products.create` untuk membuka form penambahan produk baru. Data produk ditampilkan menggunakan tabel yang memiliki kolom Nama, Harga, Jumlah Variant, dan Aksi. Pengulangan data dilakukan menggunakan `@foreach($products as $product)` sehingga setiap data produk yang dikirim dari controller akan ditampilkan secara otomatis. Nama produk diambil dari `{{ $product->name }}`, sedangkan harga produk diformat menjadi format rupiah menggunakan fungsi `number_format()`. Jumlah variant ditampilkan menggunakan `{{ $product->variants()->count() }}`, yang berfungsi menghitung total varian dari setiap produk berdasarkan relasi model.

Pada kolom aksi, terdapat beberapa tombol untuk mengelola data produk. Tombol “Kelola Variant” digunakan untuk membuka halaman pengelolaan varian produk melalui route `variants.index`, sedangkan tombol “Edit” digunakan untuk membuka halaman edit produk melalui route `products.edit`. Selain itu, terdapat form untuk menghapus produk menggunakan method POST yang dikombinasikan dengan `@method('DELETE')` agar dikenali Laravel sebagai request DELETE. Token `@csrf` digunakan untuk keamanan agar terhindar dari serangan CSRF. Sebelum data dihapus, sistem akan menampilkan konfirmasi melalui `onsubmit="return confirm('Yakin hapus?')"` sehingga pengguna harus menyetujui terlebih dahulu sebelum proses penghapusan dilakukan.